Stage 1.4.2.2 — Detecting Scanned / Non-Text PDF Pages

Exactly. We should not install OCR yet. First we'll understand and implement **Stage 1.4.2.2 — Detecting Scanned / Non-Text PDF Pages**.
The goal is to build a small inspection experiment in your existing notebook:

```
notebooks/
+-- text-loaders/
    +-- 01_data_ingestion.ipynb
```

1. What are we trying to detect?

For every PDF page, we want to answer:

```
Does this page contain useful extractable text?
             |
        +----+----+
        |         |
       YES        NO
        |         |
        v         v
 Normal PDF      OCR candidate
```

But we'll go one step further.
A page can contain some text and still be problematic. So we'll collect useful diagnostics rather than simply returning True/False.

1.4.2.2 — PDF Page Inspection

Step 1 — Create the inspection function

Add a new Markdown cell:

```
# 1.4.2.2 Detecting Scanned / Non-Text PDF Pages

The objective is to inspect every PDF page before deciding whether normal
text extraction is sufficient or OCR may be required.
```

Then add this code cell:

```
def inspect_pdf_documents(name, documents):
    """
    Inspect LangChain Documents produced from a PDF.

    Reports:
    - page number
    - extracted character count
    - whether text exists
    - source metadata
    """

    print("=" * 80)
    print(f"PDF INSPECTION: {name}")
    print("=" * 80)

    print(f"Number of pages/documents: {len(documents)}")

    for index, document in enumerate(documents, start=1):

        content = document.page_content.strip()

        print(f"\nPage {index}")
        print("-" * 80)

        print("Characters :", len(content))
        print("Has text   :", bool(content))
        print("Metadata   :", document.metadata)
```

Step 2 — Inspect the text-based PDF

Run:

```
inspect_pdf_documents(
    "Text-based PDF",
    text_documents
)
```

You should see something conceptually similar to:

```
================================================================================
PDF INSPECTION: Text-based PDF
================================================================================

Number of pages/documents: 5

Page 1
--------------------------------------------------------------------------------
Characters : *******
Has text   : True

Page 2
--------------------------------------------------------------------------------
Characters : *******
Has text   : True
```

and so on.
The exact character counts aren't important. The important observation is **Has text : True** for the pages containing extractable text.

Step 3 — Inspect the scanned PDF

Now run:

```
inspect_pdf_documents(
    "Scanned PDF",
    scanned_documents
)
```

You should see something similar to:

```
================================================================================
PDF INSPECTION: Scanned PDF
================================================================================

Number of pages/documents: 1

Page 1
--------------------------------------------------------------------------------
Characters : 0
Has text   : False
```

This is exactly what we want to demonstrate. We have now programmatically detected:

```
Text PDF

Page 1 -> text available
Page 2 -> text available
...
versus:

Scanned PDF

Page 1 -> no extractable text
```

Step 4 — Make the detection reusable

The previous function is useful for learning, but let's improve it.
Create:

```
def analyze_pdf_pages(documents):
    """
    Analyze PDF pages and return page-level diagnostics.
    """

    results = []

    for page_number, document in enumerate(documents, start=1):

        content = document.page_content.strip()

        results.append({
            "page": page_number,
            "characters": len(content),
            "has_text": bool(content),
            "metadata": document.metadata
        })

    return results
```

Now execute:

```
text_analysis = analyze_pdf_pages(text_documents)

scanned_analysis = analyze_pdf_pages(scanned_documents)
```

Inspect text_analysis and scanned_analysis. You'll get Python dictionaries representing each page.

Step 5 — Make the result easier to understand

Because you're using a notebook, let's use a DataFrame:

```
import pandas as pd

text_df = pd.DataFrame(text_analysis)
text_df
```

page	characters	has_text
1	850	True
2	720	True
3	940	True
4	650	True
5	810	True

Now:

```
scanned_df = pd.DataFrame(scanned_analysis)
scanned_df
```

page	characters	has_text
1	0	False

This makes the difference very obvious.

Step 6 — Don't stop at has_text

This is an important production-level concept. Consider this PDF page:

```
Page 3

Characters: 37
Has text: True
```

Does that automatically mean the page is fine? No. It could be:

```
                  Page 3
                    |
               Has some text
                    |
          +---------+----------+
          |                    |
          v                    v
   Useful extraction       Poor extraction
          |                    |
          v                    v
       Continue             Investigate
```

For example, the page could contain: a large table, an image containing text, a scanned signature, a diagram, a two-column layout, or text extracted in the wrong order.

Therefore: Text presence is a detection signal, not an extraction-quality guarantee.

Step 7 — Introduce a text threshold

Let's create a simple learning-oriented classifier:

```
def classify_page(content, minimum_characters=50):
    """
    Simple learning-oriented classification.

    Returns:
        TEXT_AVAILABLE
        POSSIBLE_SCANNED
    """

    character_count = len(content.strip())

    if character_count >= minimum_characters:
        return "TEXT_AVAILABLE"

    return "POSSIBLE_SCANNED"
```

Now test it:

```
for document in text_documents:
    result = classify_page(document.page_content)
    print(result)

for document in scanned_documents:
    result = classify_page(document.page_content)
    print(result)
```

Your scanned page should be classified as: **POSSIBLE_SCANNED**

Step 8 — Why do we call it POSSIBLE_SCANNED?

This naming is intentional. We should not say 'No text -> definitely scanned PDF' because there are other possibilities:

```
No extracted text
       |
       +-- Scanned page
       |
       +-- Image-only page
       |
       +-- Extraction failure
       |
       +-- Unsupported encoding
       |
       +-- Corrupted/unusual PDF
```

Therefore our ingestion pipeline should say **POSSIBLE_SCANNED** rather than **DEFINITELY_SCANNED**. This is a much better engineering mindset.

Step 9 — Build our first PDF inspection report

Let's combine everything:

```
def generate_pdf_inspection_report(documents, minimum_characters=50):

    report = []

    for page_number, document in enumerate(documents, start=1):

        content = document.page_content.strip()
        character_count = len(content)

        if character_count >= minimum_characters:
            status = "TEXT_AVAILABLE"
        else:
            status = "POSSIBLE_SCANNED"

        report.append({
            "page": page_number,
            "characters": character_count,
            "status": status,
            "source": document.metadata.get("source"),
        })

    return report
```

Run:

```
report = generate_pdf_inspection_report(scanned_documents)
pd.DataFrame(report)
```

page	characters	status	source
1	0	POSSIBLE_SCANNED	...

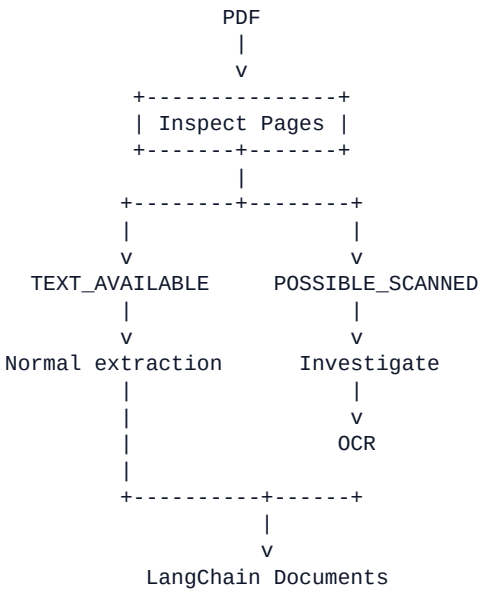
Now test the normal PDF:

```
report = generate_pdf_inspection_report(text_documents)
pd.DataFrame(report)
```

You should see **TEXT_AVAILABLE** for the normal pages.

Step 10 — Our first ingestion decision engine

We can now represent our current learning architecture as:



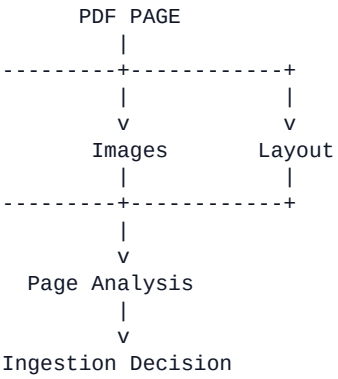
Important: We aren't actually running OCR yet. We're only building the decision point that tells us: *'This page probably needs additional processing.'*

Step 11 — One important improvement

Our current detector only examines document.page_content. For complex PDFs, that's not enough. For our next experiment we'll inspect the PDF itself:

```
Page
 |
 +-- Text blocks
 +-- Images
 +-- Image count
 +-- Text character count
 +-- Text density
 +-- Potentially suspicious pages
```

That gives us a much stronger picture:



That will be our next experiment before OCR.

Your learning progression is now:

```
1.4.2 OCR & Complex PDF Ingestion
        |
        +-- 1.4.2.1 Text PDF vs Scanned PDF       [x]
        |
        +-- 1.4.2.2 Detect Non-Text Pages         <- We are here
        |
        +-- 1.4.2.3 Deep PDF Page Analysis
        |
        +-- 1.4.2.4 OCR Fundamentals
        |
        +-- 1.4.2.5 OCR Implementation
        |
        +-- 1.4.2.6 OCR -> LangChain Documents
```

Don't install an OCR engine yet. First complete the page-analysis experiment; it will make the reason for OCR much clearer and will give you a more production-oriented mental model of document ingestion.